

Soft Voting Robustness in Neural Network Ensembles with Empirical Analysis and Formal Verification

Ádám Kovács, Roland Gunics, Gergely Kovásznai

Eszterházy Károly Catholic University
Faculty of Informatics
Department of Computational Science

September 22, 2026

Motivation

- ▶ Safety-critical AI requires more than high test accuracy: it also needs guarantees about behavior under failures.
- ▶ Neural-network ensembles can improve accuracy and stability by aggregating multiple models.
- ▶ Empirical robustness (e.g. noise or adversarial robustness) does not by itself provide a formal guarantee.
- ▶ This work studies whether an ensemble can remain stable when *one member is corrupted or removed*.

Central idea: quantify *voting robustness* and combine empirical evaluation with formal verification.

Hard Voting vs. Soft Voting

Hard voting

$$\hat{y} = \arg \max_j \sum_i 1[\hat{y}_i = j]$$

- ▶ Discrete model outputs.
- ▶ Combinatorial voting patterns.
- ▶ Difficult to encode and verify at scale.

Soft voting

$$S_j(x) = \frac{1}{n} \sum_{i=1}^n p_{ij}(x),$$
$$\hat{y}(x) = \arg \max_j S_j(x)$$

- ▶ Continuous outputs are averaged.
- ▶ Aggregation is linear.
- ▶ Can be represented as an additional affine layer.

Voting Robustness and Evaluation Metrics

Voting robustness: the minimum amount of model-level change required to alter the ensemble decision.

Metric	Interpretation
Accuracy	Baseline predictive performance.
Accuracy drop	Degradation after corruption/removal.
Margin	Gap between the winning and runner-up ensemble probabilities.
CSP	Probability that a model change causes a class switch.
LOO drop	Accuracy impact of removing one member.

From the Margin Condition to Formal Verification

The verification question is existential:

$\exists x$: the corrupted ensemble changes the predicted class?

UNSAT

- ▶ No such input exists within the encoded problem.
- ▶ The ensemble is formally robust for that verification task.

SAT

- ▶ A counterexample input exists.
- ▶ The failure can change the ensemble decision.

Formalization of Soft Voting

For an ensemble of n networks with logits o_{ij} , introduce one summed-output variable s_j per class:

Soft voting:

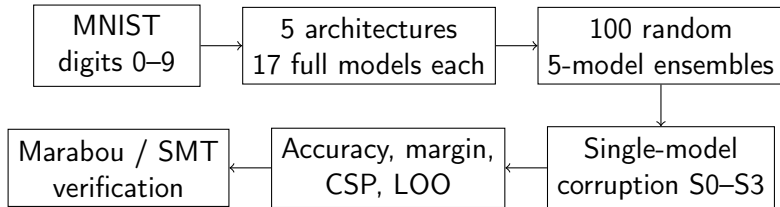
$$\sum_{i=1}^n o_{ij} = s_j.$$

The division by n is unnecessary when only the ordering of class scores matters.

Distinct Argmax outputs:

$$\bigvee_{i,j=1}^n \left(\left(\bigwedge_{k=1}^n (s_i \geq s_k) \right) \wedge s'_i < s'_j \right)$$

Experimental Design



All models are evaluated on the same MNIST test inputs, with randomness fixed for reproducibility. Trained models are exported to ONNX for verification.

Model Families and Corruption Schemes

Architecture	Parameters	Structure
SimpleLinear	7,850	$784 \rightarrow 10$
HiddenMLP	50,370	$784 \rightarrow 64 \rightarrow 10$
SingleConv	$\approx 3,074$	3×3 conv, 32 filters $\rightarrow 10$
TinyCNN	1,498	3×3 conv, 16 filters + dropout $\rightarrow 10$
CompressedModel	25,370	$784 \rightarrow 32 \rightarrow 10$

For each architecture:

- ▶ 17 independently trained full models.
- ▶ 28 corrupted models total: 7 for each corruption scheme.

Corruption schemes:

$$S_0 = \{0, 1, 2\}, \quad S_1 = \{1, 2, 3\}, \quad S_2 = \{2, 3, 4\}, \quad S_3 = \{3, 4, 5\}$$

Each corrupted model is trained without the three listed digit classes.

Formal Verification Setup

Verification engine: Marabou 2.0.

Encoding details

- ▶ Softmax is removed because it is not supported by Marabou's encoding used here.
- ▶ Raw logits are aggregated directly.
- ▶ Argmax differences are encoded as disjunctive constraints.
- ▶ Strict inequalities are replaced by non-strict inequalities using $\epsilon = 0.001$.

Three verification tasks, 50 ensembles per architecture:

1. Model Failure: full vs. corrupted ensemble.
2. Leave-One-Out: corrupted ensemble vs. ensemble with the corrupted model removed.
3. Ensemble Equivalence: compare two corrupted ensemble instances.

Timeout: **1200** s per instance.

Formal Verification: Problem Size

Model / task	#Vars	ReLU	Equations/inequalities
SimpleLinear / Failure	864	0	80
SimpleLinear / LOO	854	0	70
SimpleLinear / Equivalence	844	0	60
HiddenMLP / Failure	1,632	384	464
HiddenMLP / LOO	1,494	320	390
HiddenMLP / Equivalence	1,484	320	380
CompressedModel / Failure	1,248	192	272
CompressedModel / LOO	1,174	160	230
CompressedModel / Equivalence	1,164	160	220
SingleConv / Failure	260,448	129,792	129,872
TinyCNN / Failure	130,656	64,896	64,976

The convolutional models produce dramatically larger verification instances than the linear/MLP models.

Verification Results: SimpleLinear

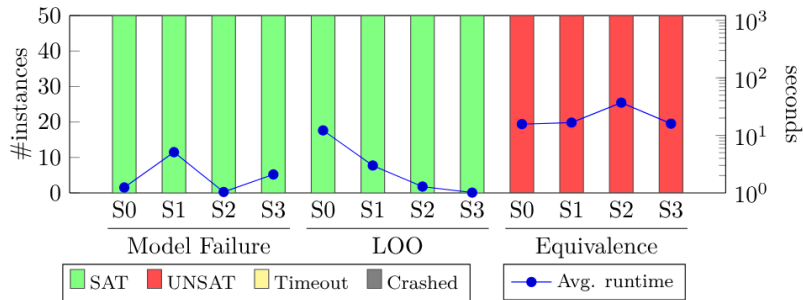


Figure: Verification results for **SimpleLinear** ensembles.

Verification Results: HiddenMLP

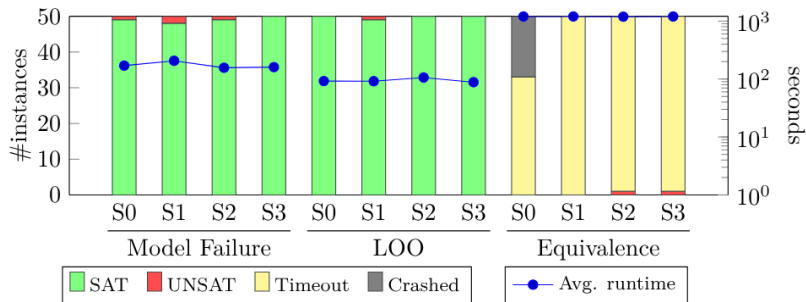


Figure: Verification results for **HiddenMLP** ensembles.

Verification Results: CompressedModel

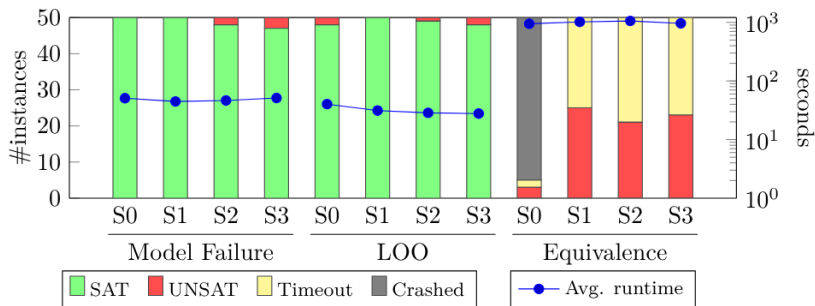


Figure: Verification results for **CompressedModel** ensembles.

Limitations and Future Work

Limitations reported in the thesis

- ▶ Controlled MNIST classification setting.
- ▶ Focus on single-model failures.
- ▶ Formal verification struggled with deeper/convolutional architectures.
- ▶ Precision handling in Argmax comparison introduced a small source of verification error.

Natural next steps

- ▶ Verify multiple simultaneous model failures.
- ▶ Scale to larger, more realistic datasets and ensemble sizes.
- ▶ Improve verifier memory usage and problem-specific heuristics.
- ▶ Investigate how maximizing ensemble margin can improve both robustness and certifiability.

References



A. Kovacs et al., "Soft voting robustness in neural network ensembles with empirical analysis and formal verification," *Annales Mathematicae et Informaticae*, 2025



L. Antal, E. Abraham, H. Masara, "Generalizing neural network verification to the family of piece-wise linear activation functions," *Science of Computer Programming*, 243 (2025).



R. Bunel et al., "A Unified View of Piecewise Linear Neural Network Verification," *NeurIPS*, 2018.



D. Gross et al., "Robustness Verification for Classifier Ensembles," *ATVA*, 2020.



G. Katz et al., "The Marabou Framework for Verification and Analysis of Deep Neural Networks," *CAV*, 2019.



Y. LeCun et al., "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, 86(11), 1998.



T. Pang et al., "Improving adversarial robustness via promoting ensemble diversity," *ICML*, 2019.



G. Singh et al., "An Abstract Domain for Certifying Neural Networks," *POPL*, 2019.

Thank you

Questions?

Ensemble Construction and Evaluation

- ▶ Construct 100 ensembles per architecture by randomly selecting 5 of the 17 full models.
- ▶ Replace exactly one ensemble member with a corrupted model from the same architecture.
- ▶ Create four corrupted variants per original ensemble (S0–S3).
- ▶ Aggregate model outputs with soft voting.
- ▶ Evaluate accuracy, accuracy drop, margin, CSP, and leave-one-out impact.

Pipeline implementation:

- ▶ TensorFlow/Keras for model training.
- ▶ NumPy/Python for ensemble statistics and reproducibility.
- ▶ ONNX export for interoperability.
- ▶ Marabou 2.0 for SMT-based formal verification.

Selected Empirical Results

Model	Corr.	Acc.	Acc. Drop	Margin	CSP
SimpleLinear	–	0.92570	0.00000	0.84442	0.00000
SimpleLinear	S1	0.92410	0.00160	0.75020	0.01130
HiddenMLP	–	0.97430	0.00000	0.95354	0.00000
HiddenMLP	S1	0.97490	-0.00060	0.84143	0.00310
SingleConv	–	0.98148	0.00000	0.97727	0.00005
SingleConv	S1	0.98216	-0.00067	0.86128	0.00263
TinyCNN	–	0.98232	0.00000	0.96820	0.00005
TinyCNN	S1	0.98319	-0.00088	0.85450	0.00304
CompressedModel	–	0.96500	0.00000	0.93244	0.00000
CompressedModel	S1	0.96550	-0.00050	0.82351	0.00550

Negative accuracy-drop values mean that a corrupted ensemble happened to score slightly higher than its corresponding baseline in that trial condition; the thesis attributes these rare cases to dataset characteristics and random variation.